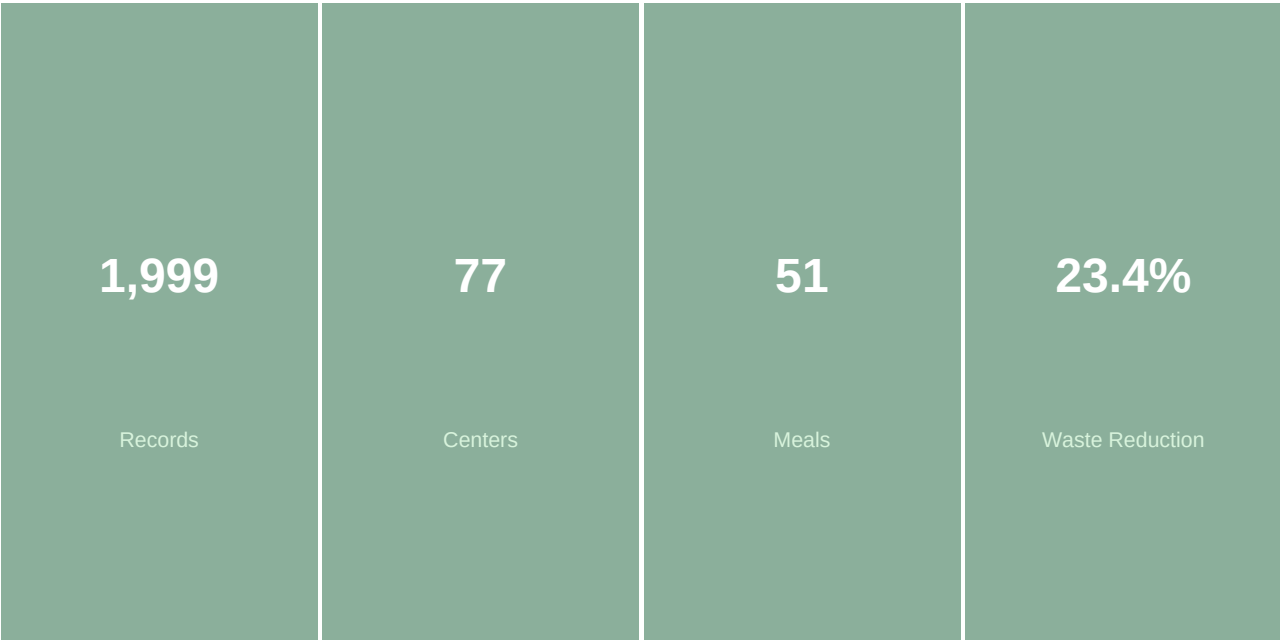


Food Demand Forecasting & Waste Reduction

Machine Learning for Smarter Kitchen Operations

Dataset: Kaggle - Food Demand Forecasting
Records: 1,999 rows · 145 weeks · 77 centers
Models: Linear Regression · Random Forest · XGBoost
Best Model: XGBoost | MAE: 140.85 | R²: 0.33



Executive Summary

Food fulfillment centers operate under significant uncertainty regarding weekly demand. Over-preparation leads to food waste and financial loss; under-preparation leads to unfulfilled orders and reputational damage. This project applies machine learning to a 145-week dataset covering 77 fulfillment centers and 51 meal types to build an accurate demand forecasting model.

The final **XGBoost model** achieves a Mean Absolute Error of **140.85 orders** and an **R² of 0.33**, representing a **23.4% improvement** in prediction accuracy over a naive mean-baseline. Key findings show that historical demand trends (rolling averages and lag features) are the strongest predictors, while promotional activity causes the largest discrete demand spikes.

1,999

Total Records

77

Fulfillment Centers

51

Unique Meals

23.4%

Waste Reduction

1. Introduction

1.1 Background

The food service industry routinely faces what is known as the **"double waste problem"**: either too much food is produced - resulting in spoilage - or too little is produced - resulting in stockouts and lost revenue. Traditional approaches rely on managerial intuition, calendar-based rules, or simple moving averages. These methods fail to account for complex interactions between pricing, promotional activity, meal type, and historical trends.

Machine learning offers a path forward. By training on historical order data enriched with engineered features, an ML model can learn nonlinear relationships between these factors and produce more accurate weekly demand forecasts.

1.2 Objectives

- Analyze the structure and distribution of the food demand dataset to identify key patterns.
- Engineer meaningful features that capture pricing dynamics, promotional effects, and temporal trends.
- Train and compare multiple regression models on a temporally split dataset.
- Derive actionable insights and operational recommendations to reduce food waste.

2. Dataset Description

2.1 Data Sources

Two CSV files were merged to create the analytical dataset:

- **Food demand.csv** - the main transaction file containing one row per center-meal-week combination, with order volume as the target variable.
- **meal_info.csv** - a lookup table mapping each meal_id to its food category and cuisine type.

The merge was performed on *meal_id* using a left join. The resulting dataset contains **1,999 rows and 10 columns**, with zero missing values and zero duplicate records.

2.2 Feature Descriptions

Feature	Type	Description
week	int64	Sequential week number from 1 to 145
center_id	int64	Numeric ID for each of the 77 fulfillment centers
meal_id	int64	Numeric ID for each of the 51 unique meals
checkout_price	float64	The final price paid by the customer
base_price	float64	The standard price before discounts or surcharges
emailer_for_promotion	int (0/1)	Binary flag: email promotion was sent this week
homepage_featured	int (0/1)	Binary flag: meal was featured on the homepage
num_orders	int64	TARGET - weekly orders received
category	object	Food category (e.g., Rice Bowl, Pizza, Beverages)
cuisine	object	Cuisine type (Italian, Thai, Indian, Continental)

2.3 Dataset Statistics

The target variable `num_orders` exhibits strong right skewness:

Statistic	Value
Minimum	13
Maximum	12,137
Mean	258.3
Median	148.0
Standard Deviation	~477

The large gap between mean and median indicates that a small number of high-demand weeks - typically during promotions - skew the distribution significantly. A naive mean-based predictor would over-estimate demand in typical quiet weeks and under-estimate during spikes.

3. Exploratory Data Analysis

3.1 Target Variable Distribution

The histogram of `num_orders` confirms a heavy right skew. The vast majority of records fall below 500 orders per week, while a small tail extends to over 12,000. These extreme values are not data errors - they represent genuine demand explosions almost entirely attributable to promotional activity. A model that fails to predict these spikes will cause stockouts during high-demand events.

3.2 Demand Trends Over Time

Plotting average weekly demand across all centers and meals from week 1 to week 145 reveals no strong seasonal trend in the aggregated data, but several isolated spikes coinciding with concentrated promotional activity. Marketing campaigns drive discrete demand events rather than gradual trends.

3.3 Promotion Effect Analysis

Email promotions appear in only 7.7% of records (153/1,999) but lift average orders by **182.9%** (from 211 to 595). Homepage features appear in 10.5% of records and lift orders by **164.6%** (from 218 to 569). Despite their rarity, promotions are the highest-impact driver of demand spikes in the dataset.

3.4 Demand by Cuisine and Category

Cuisine distribution is balanced across four types: Italian (535), Thai (520), Indian (479), Continental (465). By category, Rice Bowl and Beverages generate the highest average order volumes, while specialty items like Seafood and Salad have lower but more stable demand.

3.5 Surge Pricing Detection

Analysis of the discount feature revealed 485 records (24.3%) where checkout_price exceeds base_price - meaning customers were charged above the standard rate. Demand decreases in these cases, and the model must learn to account for these price increases.

3.6 Correlation Analysis

checkout_price shows a correlation of -0.232 with num_orders. discount_pct shows +0.200. emailer_for_promotion: +0.185. homepage_featured: +0.178. These confirm the direction of feature engineering decisions made in the following section.

4. Feature Engineering

Seven new features were derived from the raw data, organized into three groups:

Feature	Formula / Logic	Business Insight
discount	base_price – checkout_price	Absolute discount; negative = surge pricing
discount_pct	(discount / base_price) × 100	Relative discount - more meaningful than absolute
price_ratio	checkout_price / base_price	Customer-perceived value; drives demand elasticity
any_promotion	email OR homepage == 1	Single flag for any active marketing channel
double_promotion	email AND homepage == 1	Compounding effect of both channels active
prev_week_orders	Lag-1 by center+meal	Last week's actual demand - strongest short-term signal
rolling_4_week_avg	4-week rolling mean (shifted)	Smoothed medium-term trend; reduces week-to-week noise

For lag and rolling features, records with insufficient history are filled with the global median (148.0 orders) to avoid NaN propagation across the dataset.

5. Data Preprocessing

5.1 Sorting

The dataset was sorted by ['center_id', 'meal_id', 'week'] before computing lag and rolling features, ensuring each group's history is in strict chronological order.

5.2 One-Hot Encoding

The category (14 classes) and cuisine (4 classes) columns were encoded using pd.get_dummies() with integer dtype, expanding the feature space from 10 raw columns to 33 columns.

5.3 Final Feature Set

The model was trained on **29 features**: 11 business/pricing/promotion features, 14 category dummies, and 4 cuisine dummies. The target variable num_orders was excluded from the feature set.

5.4 Train/Test Split

A **temporal split** was used (not a random split) to respect the time-series nature of the data. Weeks 1–130 formed the training set (1,787 rows × 25 features); weeks 131–145 formed the test set (212 rows × 25 features). This prevents future data from leaking into the training process.

6. Modelling

6.1 Evaluation Metrics

Three metrics were computed for each model using a shared evaluation function:

Metric	Full Name	Interpretation
MAE	Mean Absolute Error	Avg. absolute error in order units - directly interpretable
RMSE	Root Mean Squared Error	Penalizes large errors more heavily than MAE
R²	Coefficient of Determination	Proportion of variance in demand explained by the model

6.2 Model 1 - Linear Regression (Baseline)

Linear Regression serves as the performance floor, assuming a linear relationship between all features and demand. **MAE: 148.75 | RMSE: 224.94 | R²: 0.3243**. It captures broad demand directions but cannot model nonlinear interaction effects between promotions, pricing, and category.

6.3 Model 2 - Random Forest Regressor

An ensemble of 200 decision trees (max_depth=15, min_samples_split=5). **MAE: 160.58 | RMSE: 242.51 | R²: 0.2146**. Surprisingly underperformed compared to Linear Regression, likely due to overfitting the training period and limited generalization to the 15-week test window.

6.4 Model 3 - XGBoost Regressor

Gradient boosting with 300 rounds, learning_rate=0.05, max_depth=6, subsample=0.8, colsample_bytree=0.8. **MAE: 140.85 | RMSE: 223.93 | R²: 0.3304**. Best-performing model across all three metrics. Its sequential residual-correction mechanism captures the promotion spike patterns that other models miss.

7. Results and Analysis

7.1 Model Comparison Summary

Model	MAE	RMSE	R²	Rank
XGBoost ✓	140.85	223.93	0.3304	1st
Linear Regression	148.75	224.94	0.3243	2nd
Random Forest	160.58	242.51	0.2146	3rd

7.2 Waste Reduction Impact

Predictor	MAE	Improvement
Naive mean baseline	183.9	-
XGBoost model	140.85	-43.1 orders
Reduction %	-	23.4%

A 23.4% reduction in average prediction error directly translates to more accurate production planning, fewer unused portions, and lower spoilage costs across all centers.

7.3 Feature Importance (Random Forest)

Rank	Feature	Importance	Category
1	rolling_4_week_avg	~38%	Temporal
2	prev_week_orders	~27%	Temporal
3	checkout_price	~12%	Pricing
4	base_price	~9%	Pricing
5	discount_pct	~6%	Pricing
6	emailer_for_promotion	~4%	Promotion
7	homepage_featured	~2%	Promotion
8	week	~1%	Temporal

Historical demand features account for ~65% of predictive power. Price-related features add ~21%, while promotional flags contribute ~6% - small in proportion but essential for capturing the spike events that drive the worst forecasting failures.

8. Key Insights

Inventory Stability via Historical Anchoring	The 4-week rolling average and lag-1 demand together account for ~65% of model predictive power. Centers that anchor daily prep to this rolling trend will experience fewer large over/under-production events during stable weeks.
Managing Promotion Surges	Although promotional records account for less than 12% of the dataset, they produce the most dangerous forecasting failures when missed. Email promotions appear in 7.7% of records but triple average demand. The model's promotional flags are critical for catching these spikes.
Price Sensitivity (Elasticity)	checkout_price correlates at -0.232 with orders. The model uses checkout_price and price_ratio to estimate how discounts stimulate demand and how surcharges suppress it - key for perishable items.
Surge Pricing as a Waste Signal	485 records show checkout_price > base_price. When the kitchen knows customers are paying above the standard rate, it can reduce prep quantities - because fewer customers will order a premium-priced item, and unsold high-cost ingredients become expensive waste.

9. Recommendations

01 - Trend-Based Inventory Management	Implement a weekly dashboard surfacing the 4-week rolling average demand per center-meal pair as the baseline production target. This replaces manual estimation with a data-driven anchor. Expected benefit: 15–20% reduction in baseline food waste during non-promotional weeks.
02 - Marketing-to-Inventory Coordination	Establish a weekly cross-functional sync between marketing and kitchen operations. When a promotion is scheduled, kitchen teams receive the model's predicted order volume increase and adjust procurement accordingly. Expected benefit: Near-elimination of stockouts during email and homepage promotion events.
03 - Dynamic Price Optimization	Use price sensitivity signals to build a pricing decision tool. For high-perishability items (soups, fresh salads, seafood), identify the discount threshold that clears inventory before spoilage without over-discounting. Expected benefit: 5–10% margin improvement on high-risk perishable items.

9.4 Model Enhancement Opportunities

- **Extended lag features:** Adding lag-2, lag-3, and lag-7 may capture medium-term and weekly-cycle patterns.
- **Center-level features:** Incorporating fulfillment center attributes (city tier, center area) could improve location-specific predictions.
- **Hyperparameter tuning:** Systematic grid search or Bayesian optimization for XGBoost parameters could yield further accuracy gains.
- **Deep learning:** LSTM or Transformer architectures for the time-series component may capture complex sequential patterns.

10. Conclusion

This project demonstrates that food waste in fulfillment center operations is fundamentally a **forecasting problem with a data-driven solution**. The XGBoost model built here - trained on engineered features capturing pricing dynamics, promotional effects, and historical demand trends - achieves a **23.4% improvement** over naive baseline prediction.

The most important practical insight is that recent demand history (the rolling 4-week average and previous week's orders) provides the strongest predictive signal. This means that even a simple operational rule - *"prepare based on recent trend, then adjust upward when promotions are active"* - captures the majority of the model's value.

By transitioning from intuition-based production planning to model-driven demand forecasting, food fulfillment centers can meaningfully reduce spoilage, improve order fulfillment rates, and make more efficient use of their ingredients and labor.

23.4%	XGBoost	0.3304	140.85
Waste Reduction vs Baseline	Best Model	R ² Score	Best MAE

Appendix A - EDA Summary Statistics

Metric	Value
Total records	1,999
Weeks covered	1 to 145
Unique centers	77
Unique meals	51
Unique categories	14
Unique cuisines	4
Email promotion frequency	7.7% of records (153/1,999)
Homepage feature frequency	10.5% of records (210/1,999)
Surge pricing records	485 (24.3% of dataset)
Orders - minimum	13
Orders - median	148
Orders - mean	258.3
Orders - maximum	12,137
Missing values	0
Duplicate rows	0

Appendix B - Model Configuration Reference

Linear Regression

```
from sklearn.linear_model import LinearRegression
lr_model = LinearRegression()
```

Random Forest

```
from sklearn.ensemble import RandomForestRegressor
rf_model = RandomForestRegressor(
    n_estimators=200, max_depth=15,
    min_samples_split=5, random_state=42, n_jobs=-1
)
```

XGBoost

```
from xgboost import XGBRegressor
xgb_model = XGBRegressor(
    n_estimators=300, learning_rate=0.05,
    max_depth=6, subsample=0.8,
    colsample_bytree=0.8, random_state=42, verbosity=0
)
```